

TÀI LIỆU ĐÀO TẠO

VIBE CODING

với AntiGravity

Buổi 01 · Khung lý thuyết & mạch tư duy

"Thin harness, fat skills" — một triết lý kiến trúc cho kỷ nguyên AI

◆ ◆ ◆
Nguyễn Minh Thi

P.CLML · TT.KTTC · VTNet | Biên soạn phục vụ khóa đào tạo nội bộ

Mục lục

Dẫn nhập – Kiềng 3 chân cho một triết lý kiến trúc	4
LM là một hệ bất định	4
“Context” là vua, “harness” là hoàng đế	4
Người tỏ - kẻ mờ	6
Phần I — Khám phá & Phát triển	7
Bài 1 — Khởi tạo dự án cùng IDE thể hệ mới	7
Bài 2 — Skills: bộ não mở rộng của AI	9
Bài 3 — Công cụ ngoài (MCP): cầu nối giữa phần mờ và phần rõ	11
Phần II — Hợp tác & Quản trị	15
Bài 4 — Sub-agent: chia việc cho đội hình AI chuyên trách	15
Bài 5 — Agent Manager và khung quản trị	16
Bài 6 — Knowledge Items: trí nhớ sống của hệ thống	18
Phần III — Vận hành toàn trình	22
Bài 7 — Dòng chảy phiên bản rảnh tay với Git	22
Bài 8 — Thử thách tổng hợp: Capstone nghiệm thu toàn hệ thống	23
Tổng kết: Từ tám bài thực hành đến một phẩm chất hệ thống	26
Nhìn lại mạch chuyện	26
Trục xương sống đã hiện rõ	26
Mở rộng ra cấp tổ chức	26
Lời nhắn cuối	27
Phụ lục A — Thuật ngữ cốt lõi	28
Phụ lục B — Nguồn tham khảo đọc thêm	30

Lời mở đầu

Có một điều đáng suy nghĩ đang diễn ra trong làn sóng trí tuệ nhân tạo hiện nay. Thứ giới hạn bạn bây giờ không còn là việc bạn có biết lập trình hay không, có thạo tiếng Anh hay không, có được đào tạo bài bản trong một chuyên ngành kỹ thuật hay không. AI đã lấp được phần lớn những khoảng trống ấy. Thứ thật sự giới hạn bạn — và cũng là thứ quyết định bạn đi được xa đến đâu — là sức sáng tạo, tư duy tổ chức, và khả năng tiếp cận, khai thác tối đa các công cụ AI hiện có.

AI không sinh giá trị từ hư không. Nó là một cỗ máy khuếch đại tư duy của chính bạn. Nếu bạn mang tới một ý tưởng rõ ràng và một tư duy có cấu trúc, AI sẽ nhân nó lên gấp mười, gấp trăm. Nếu bạn mang tới một ý tưởng mơ hồ, AI cũng sẽ khuếch đại chính sự mơ hồ đó. Và khi ai đó vừa có ý tưởng mạnh, vừa biết cách sử dụng các kỹ thuật để dắt AI đi đúng hướng, khoảng cách năng suất giữa họ và phần còn lại sẽ lớn dần theo cấp số nhân.

Giá trị cuối cùng = Ý tưởng của bạn × Chất lượng kiến trúc bao quanh AI. Tài liệu này sẽ gợi mở cho bạn phần kiến trúc hiện đại, còn phần ý tưởng hoàn toàn tùy thuộc vào bạn.

Đây là lý do khóa học tồn tại. Chúng ta không học để nhớ cú pháp của một ngôn ngữ lập trình cụ thể, không học để thuộc tên từng công cụ, càng không học để rập khuôn một quy trình. Chúng ta học để trang bị tư duy: tư duy về cách đặt AI vào đúng chỗ, tư duy về cách phân công giữa phần mờ và phần rõ, tư duy về cách chưng cất kinh nghiệm thành năng lực có thể “triệu hồi”. Khi có tư duy ấy, công cụ nào rơi vào tay bạn cũng sẽ được vận dụng đến tối đa.

Tám bài học phía sau không phải là tám thủ thuật rời rạc. Chúng là tám tầng của một kiến trúc duy nhất — kiến trúc giúp một người bình thường, trong một buổi chiều, có thể dựng lên một "nhà máy phần mềm tự động" thu nhỏ của riêng mình. Nhưng trước khi vào tầng thứ nhất, chúng ta cần 1 cái kiềng ba chân làm điểm tựa cho mọi quyết định thiết kế phía sau.

Dẫn nhập – Kiềng 3 chân cho kiến trúc về AI

LLM là một hệ bất định

Phần mềm truyền thống là một hệ xác định. Bạn đưa cùng một đầu vào, nó trả về cùng một đầu ra, không sai một ly. Đó là lý do chúng ta tin vào nó để tính thuế, chuyển tiền, hay kiểm soát lò phản ứng hạt nhân. Tính xác định là khế ước ngầm giữa máy và người.

Cho đến khi LLM xuất hiện, nó bước ra ngoài khế ước đó. Đầu vào của LLM là ngôn ngữ tự nhiên — thứ vốn mờ, đa nghĩa, và phụ thuộc ngữ cảnh. Đầu ra của nó cũng bất định: cùng một câu hỏi, hai lần chạy sẽ cho hai câu trả lời khác nhau, gần giống nhưng không trùng khớp. Và nếu đi với những ngữ cảnh khác nhau, toàn bộ đã thay đổi. Với LLM, đây không phải là lỗi. Nó là tính năng!

Tính bất định này như 2 mặt của 1 đồng xu: vừa là sức mạnh, cũng là vấn đề. Mạnh vì nó cho phép LLM hiểu được ngôn ngữ con người, xử lý được những yêu cầu chưa từng gặp, và linh hoạt như một người bạn đồng hành tận tụy. Từ khi xuất hiện, LLM thực sự đã phá bỏ mọi rào cản về đầu vào của mọi phần mềm truyền thống. Bạn muốn cung cấp đầu vào kiểu gì cũng được, miễn logic là đủ. Bạn không cần phải mở 1 ứng dụng, cuộn qua 5 tab, sổ 8 cái menu xuống chỉ để tìm và nhấn đúng vào 1 cái nút nào đó. Giờ bạn chỉ cần tả cho AI biết thứ mình muốn, và bùm! Mọi thứ đâu đó đã vào đó, như có 1 ông bụt vừa hiện ra vậy.

Vấn đề cũng từ đó mà nảy sinh. Vì một hệ tự động không thể đứng trên nền móng bất định mà vẫn hoạt động tin cậy. Chúng ta không thể đưa một chiếc xe tự lái dựa thuần túy vào một LLM loop vô tận cho đến khi “ổn”, rồi hy vọng nó đừng tông vào gốc cây vào 1 ngày đẹp trời nào đó.

Nhiệm vụ cốt lõi của mọi hệ AI-agent nghiêm túc, vì thế, có thể gói gọn trong một câu: tối ưu độ bất định, vừa đủ rộng để giữ được sự linh hoạt của LLM, và vừa đủ hẹp để đầu ra có thể tin tưởng được. Toàn bộ những gì chúng ta sẽ làm trong các bài sau đây đều là các phương tiện của cùng 1 mục đích này.

“Context” là vua, “harness” là hoàng đế

Mọi thứ LLM nhìn thấy trước khi sinh ra một câu trả lời đều là context — lịch sử hội thoại, tài liệu tham khảo, đầu ra của công cụ, và cả câu lệnh bạn vừa gõ. Prompt, ở đúng vị trí của nó, chỉ là một mảnh trong khối context ấy — mảnh gần nhất, do người dùng trực tiếp đặt vào. Hiểu được điều này sẽ thấy cả lịch sử điều

khiến LLM chính là lịch sử của một câu hỏi duy nhất: làm sao để kiểm soát cái khối context đang chảy vào mô hình? Ba thế hệ kỹ thuật dưới đây là ba lần “chúng ta” trả lời câu hỏi đó, mỗi lần ở một tầng sâu hơn.

Câu chuyện bắt đầu từ những ngày ChatGPT vừa ra mắt (2022–2023). Khi đó, cửa ngõ duy nhất để tác động vào LLM là ô nhập văn bản — không công cụ, không agent, không tài liệu đính kèm, chỉ có bạn và câu lệnh.

Cả một làn sóng mang tên “kỹ thuật prompt” bùng lên, với những mẹo vặt như “hãy suy nghĩ từng bước” hay “giả sử bạn là chuyên gia”. Nhìn lại, đó thực ra là giai đoạn người ta học cách nắn một mảnh context duy nhất — mảnh prompt — vì đơn giản là lúc đó chưa có mảnh nào khác để nắn. Prompt engineering, nói cho đúng, là context engineering khi context chỉ vừa bằng một câu hỏi.

Rồi AI-agent xuất hiện (2024–2025), và cuộc chơi thay đổi.

LLM bắt đầu tự đọc file, tự gọi công cụ, tự kéo về những đoạn thông tin mà người dùng chưa kịp đưa vào. Context không còn do người gõ thủ công nữa — nó được xây lên động trong lúc agent làm việc. Và rất nhanh, người ta nhận ra một nghịch lý: chất lượng đầu ra sụt giảm không phải vì thiếu thông tin, mà vì quá nhiều thông tin không đúng chỗ. Bài toán chuyển từ “viết câu lệnh khéo” sang “thiết kế cơ chế để đúng thông tin được nạp vào đúng thời điểm”. Đến đây, cộng đồng gọi tên được một chân lý đơn giản: điều khiển context chính là điều khiển đầu vào của LLM — và ai kiểm soát được context, người đó kiểm soát được cả hành vi của mô hình. Context là vua.

Nhưng vương quốc của context chỉ trị vì trong biên giới của một LLM. Từ 2025 trở đi, khi hệ thống có nhiều agent cùng chạy, cùng gọi công cụ, cùng ủy thác cho nhau — cái mà cộng đồng gọi là Agentic AI — câu hỏi không còn là “agent này thấy gì” mà là “cả dàn agent này vận hành ra sao”.

Làm sao để chúng không dẫm chân nhau? Làm sao để luồng phối hợp được kiểm soát? Làm sao có cơ chế nghiệm thu, rollback, bảo vệ kho mã khi có gì đó đi sai? Đến đây, chỉ điều khiển context thôi là chưa đủ — cần một bộ khung chụp lên toàn bộ dàn mô hình, quy định cái gì được phép đi vào, cái gì được phép đi ra, và thiết lập một vòng hồi tiếp để hệ thống tự chỉnh trên từng bước chạy. Bộ khung đó là harness. Nếu context là vua của một mô hình, thì harness là hoàng đế của cả vương quốc — nó không thay thế context, nó bao trùm và điều phối context cho từng mô hình dưới quyền.

Ba lớp này cộng dồn, không thay thế nhau. Harness tốt vẫn đòi hỏi context tốt ở mỗi agent; context tốt vẫn đòi hỏi prompt tốt ở mỗi lượt gọi. Càng lên cao, lớp phía dưới càng phải vững — và đó là lý do tám bài học phía sau sẽ lần lượt đi qua cả ba tầng này, theo đúng thứ tự chúng đã sinh ra trong lịch sử.

Người tổ - kẻ mờ

AI và code có ưu nhược điểm bù trừ gần như hoàn hảo. AI giỏi phần mờ — phán đoán, diễn giải, phát hiện hình mẫu, làm việc với ngôn ngữ tự nhiên. Code giỏi phần rõ — tính toán, truy vấn, kiểm tra điều kiện, thực thi lặp đi lặp lại không sai lệch. Một hệ thống tốt là hệ thống biết phân công giữa hai loại lao động này.

Nguyên tắc cho sự phân công lao động hợp lý này có tên là “Fuzzy-Front / Deterministic-Back”, và có thể phát biểu ngắn gọn như sau: “push intelligence up, push execution down” - đẩy trí tuệ lên trên, đẩy thi hành xuống dưới. Những việc cần phán đoán sẽ được làm ở tầng trước (AI). Khi mọi việc đã sáng tỏ rồi thì những việc cần kết quả ổn định, kiểm chứng được sẽ được thi hành ở tầng sau (code).

Sai lầm phổ biến nhất khi thiết kế hệ AI là lẫn lộn hai miền này. Người ta bắt LLM đi tính thuế, rồi ngạc nhiên khi kết quả có lúc lệch một (vài) chữ số. Người ta bắt LLM đi tra dữ liệu chính xác trong cơ sở dữ liệu, rồi phát hiện nó tự "bịa" ra con số nghe có vẻ hợp lý. Ngược lại, người ta nhét cả mớ logic cứng vào bên trong một agent, rồi than thở về lỗi khó debug. Đúng chỗ mới tạo ra năng suất; sai chỗ tạo ra bug mới.

Ba điều trên không phải là những nhận định riêng lẻ mà là những nguyên tắc hoạt động nền tảng cho các hệ thống AI sản xuất hiện nay, dù gọi bằng tên gì đi chăng nữa. Toàn bộ tám bài học tiếp theo sẽ xoay quanh chiếc kiềng này nhiều lần — mỗi bài là một góc nhìn mới, một công cụ mới, nhưng vẫn bắt nguồn từ cùng một nền.

Phần I — Khám phá & Phát triển

Ba bài đầu tiên này sẽ đặt nền móng cho hệ thống AI của chúng ta. Ta dựng một dự án, trang bị cho AI một "bộ não mở rộng" dưới dạng các kỹ năng được chưng cất, và nối dài cánh tay của nó ra thế giới ngoài thông qua các công cụ chuẩn. Cuối Phần I này, chúng ta sẽ có một tác nhân AI (AI agent) hiệu quả đáng kể — nhưng vẫn là một người. Cơ hội tiếp theo nằm ở việc biến một người thành một đội hình.

Bài 1 — Khởi tạo dự án cùng IDE thế hệ mới

Bối cảnh & Vấn đề

Trước khi có các IDE agent-first, khoảng cách giữa một ý tưởng và một dự án chạy được là khá lớn. Người mới học phải biết dựng dự án, cấu hình môi trường, chọn stack, viết những dòng mã nền tảng — những việc không có gì sáng tạo nhưng đủ để làm nản lòng phần đông. Ngay cả người có kinh nghiệm cũng tốn nhiều giờ cho các công việc lặp lại.

IDE thế hệ mới hướng tới việc xóa nhòa đoạn đường ấy. Bạn đưa cho nó một ý tưởng bằng ngôn ngữ tự nhiên, kèm một vài sở thích — chủ đề, phong cách thẩm mỹ, cơ chế chẳng hạn — và nó tự dựng lên một dự án có thể chạy được. Đây không phải ma thuật: đằng sau là một bộ khung (harness) đã được các kỹ sư Google xây sẵn.

Tư duy & Nguyên lý

Bài 1 là lúc ba nguyên lý nền bắt đầu lộ rõ. Việc AI có thể "hiểu" một mô tả ngôn ngữ tự nhiên và sinh ra một dự án tương ứng chính là biểu hiện của tính bất định: ở đây, bất định lại là sức mạnh — nó cho phép người không biết code vẫn khởi xướng được một phần mềm. Đồng thời, việc IDE cho AI quyền truy cập toàn codebase là một dạng context engineering nguyên thủy: ta đã nói context của mô hình từ vài đoạn chat ra cả một dự án.

Bản thân IDE là một harness — bộ khung vận hành bao quanh mô hình, lo việc gọi mô hình theo vòng lặp, quản lý context, kết nối file và công cụ. Đây là một harness mỏng đã được Google đóng gói sẵn, và tại thời điểm ta mở dự án mới, phần "kỹ năng" còn rỗng. Sức mạnh thật sự sẽ xuất hiện khi ta bắt đầu rót "cơ bắp" vào bộ khung ấy ở các bài sau.

Cách tiếp cận

Quy trình khởi tạo một dự án trên IDE agent-first thường đi qua ba bước. **Thứ nhất**, bạn cung cấp cho AI các yêu cầu đầu vào cơ bản — không nhiều, chỉ cần vừa đủ để có một "xương sống" yêu cầu. Đừng quá lo lắng về việc yêu cầu của bạn không đủ tốt hay rõ ràng. Nếu AI không biết làm gì, nó sẽ hỏi. Nếu nó vẫn làm khi chưa rõ, và đi chệch hướng, bạn hoàn toàn có thể dừng nó lại bất cứ lúc nào và sửa chữa/bổ sung. **Thứ hai**, nó lập một Implementation Plan và trình ra để người dùng duyệt — đây là artifact đầu tiên, và cũng là nơi ta lần đầu thấy AI suy nghĩ trước khi hành động. Thứ ba, sau khi được bạn phê duyệt, nó mới sinh mã, dựng cấu trúc thư mục, và chạy thử.

Ở mức kiến trúc, quy trình này chính là Fuzzy-Front / Deterministic-Back trong một vòng nhỏ: AI lo phần mờ (hiểu ý định, lập kế hoạch, viết code), còn môi trường — server dev, trình duyệt, hệ thống file — đảm nhiệm phần thi hành xác định.

Ví dụ một dự án web đơn giản hoàn toàn có thể chạy với HTML/CSS/JavaScript thuần và một server tĩnh. Việc chọn ngôn ngữ/stack nào không phải trọng tâm — trọng tâm là học viên nhìn thấy được một vòng tròn khởi tạo hoàn chỉnh: ý tưởng → kế hoạch → mã → sản phẩm chạy được.

Lợi ích thu được

- Rào cản "biết gõ code" nay đã được hạ xuống: ý tưởng có thể trở thành phần mềm mà không cần đi qua cú pháp.
- Artifact *Implementation Plan* cho người dùng khả năng phê duyệt trước khi AI hành động — giảm bất định đầu ra bằng cách chèn một khoảng khắc kiểm soát ngay đầu chuỗi.
- Dự án sinh ra đã có cấu trúc và tài liệu nền (README, ARCHITECTURE), tạo bộ phóng cho các bài sau.

Giới hạn còn lại

Điều ta có ở cuối Bài 1 là một AI biết code tốt theo mặt bằng chung. Nhưng "mặt bằng chung" này chưa đủ. Mỗi dự án, mỗi tổ chức đều có phong cách riêng, có những quy tắc đã được trả giá bằng nhiều lần thất bại mà không sách giáo khoa nào ghi lại. AI không biết những điều đó, và bản thân harness của IDE cũng không thể chứa tất cả. Nút thất kế tiếp lộ ra: làm sao trang bị cho AI một "bộ não mở rộng" chứa tri thức riêng của ta mà không “hạ bệ” nhà vua context? Đó là lý do ra đời cho Bài 2.

Bài 2 — Skills: bộ não mở rộng của AI

Bối cảnh & Vấn đề

Khi đã có một dự án chạy được, câu hỏi tiếp theo luôn là: làm sao để AI làm việc theo cách chúng ta muốn, chứ không phải theo cách mà nó muốn? Mỗi đội ngũ có quy ước riêng về code sạch, có cách debug có hệ thống, có chuẩn lập kế hoạch, có nguyên tắc kiểm tra trước khi giao riêng... Nếu phải nhắc lại tất cả những điều đó trong từng cuộc hội thoại sẽ thật là phiền phức. Và thực tế không ai làm vậy, chúng ta sẽ mắng mỏ AI sao lại không nhớ điều mà ta đã nhắc nó ở 10 prompt trước, rồi làm bầm nhắc lại và bảo nó “phải nhớ”.

Cách tiếp cận ngây thơ là viết một "mega-prompt" thật dài, rồi dán nó vào đầu mỗi phiên làm việc. Cách này sẽ thất bại nhanh chóng: context bị ngập, AI mất tập trung vào yêu cầu hiện tại, và càng nhiều chỉ dẫn thì càng nhiều chỗ để AI diễn giải sai.

Tư duy & Nguyên lý

Đây là bài đầu tiên mà một triết lý kiến trúc quan trọng hiện ra trần trụi, được Garry Tan gói gọn trong một cụm rất đáng nhớ: thin harness, fat skills — khung mỏng, kỹ năng dày. Harness là bộ khung vận hành đã gặp ở Bài 1. Skills là những năng lực được đóng gói — quy trình, phương pháp, khung suy nghĩ, thủ tục nghiệp vụ — mà mô hình có thể tái sử dụng nhiều lần. Rất nhiều hệ thống đi theo hướng ngược lại: bộ khung phình ra, đầy công cụ và logic cứng, trong khi phần kỹ năng thực sự lại nghèo nàn, ngắn hạn. Kết quả là mô hình bị ngợp trong một đám dây nhợ, context đầy thông tin thừa, còn tri thức vận hành thì không có, hoặc tản mát khắp nơi. Lựa chọn kỷ luật hơn là ngược lại: bộ khung đủ gọn để không chiếm mất sự chú ý của mô hình, phần kỹ năng thật dày — tri thức, quy trình, nguyên tắc đều được tích lũy, chưng cất, đóng gói thành các đơn vị có thể gọi lại.

Vấn đề không phải là làm AI "nhớ" nhiều hơn, mà là làm AI có thể truy hồi đúng phần tri thức vào đúng thời điểm. Giải pháp có hai nửa.

Nửa thứ nhất: biến từng mảng tri thức thành một kỹ năng (skill) độc lập, được chưng cất thành năng lực đóng gói. Ở đây cần phân biệt rõ: skill $\neq$ prompt. Một prompt là câu lệnh hiện tại — bảo AI làm gì ngay lúc này. Một skill gắn với một phương pháp hơn là một chỉ thị: nó mô tả cách tiếp cận, trình tự tư duy, các bước điều tra, tiêu chí đánh giá, những cảnh báo đã rút ra từ lần thất bại trước. Người dùng mang tới bài toán; skill mang tới quá trình giải toán (chứ không phải lời

giải). Khi một tổ chức biết chưng cất kinh nghiệm thành skill, họ không còn dùng AI theo kiểu gõ câu thần chú; họ đang dần huấn luyện hệ thống làm việc theo phong cách của chính mình.

Nửa thứ hai: có một mục lục skill — một resolver nguyên thủy — để AI không cần đọc hết mọi skill mỗi lần. Có hai khái niệm cần làm rõ ở đây. Ngân sách chú ý (attention budget): cửa sổ ngữ cảnh là tài nguyên hữu hạn, và trong tài nguyên đó, sự chú ý lại là thứ đắt nhất. Đổ toàn bộ tri thức vào context không làm mô hình thông minh hơn — mà ngược lại, tín hiệu sẽ bị pha loãng trong nhiễu. Resolver là cơ chế định tuyến: nó quyết định lúc nào nên nạp tài liệu nào, trong tình huống nào sẽ đọc skill nào trước. Resolver bảo mô hình nên đọc skill nào, còn skill bảo mô hình nên làm thế nào trong trường hợp này. Một hệ trưởng thành không mang mọi thứ trong đầu cùng lúc; nó có bản đồ, có chỉ mục, và chỉ truy hồi khi có tín hiệu phù hợp. Ký ức hữu ích không phải ký ức chứa tất cả, mà là ký ức hữu dụng đúng lúc.

Mỗi skill được tạo ra là một mảnh tư duy được đóng gói. Từ đây, AI sẽ khuếch đại mảnh tư duy ấy vô số lần. Skill dày nghĩa là bạn đang trao cho AI một phần năng lực để nó nhân bản.

Cách tiếp cận

Hệ sinh thái skill trưởng thành có ba đặc trưng. Thứ nhất, mỗi skill được viết dưới dạng một tài liệu có cấu trúc (chứ không phải một đoạn text tự do), thường có phần mô tả triggers, phần mô tả quy trình, và phần ví dụ/ngoại lệ. Thứ hai, skill được lưu trữ ở một nơi chuẩn, và việc AI truy cập skill đi qua một lớp resolver — tra mục lục trước, nạp nội dung sau. Thứ ba, số lượng skill được kiểm soát chặt: thà ít mà đúng, còn hơn nhiều mà loãng, vì mỗi skill trong mục lục đều tiêu tốn một phần attention.

Về nguồn gốc, skill có thể đến từ nhiều chỗ: cộng đồng mã nguồn mở chia sẻ các skill thông dụng (như chuẩn code sạch, debug có hệ thống, lập kế hoạch ngắn gọn), tổ chức tự viết các skill đặc thù của mình, hoặc cá nhân tích lũy qua thời gian. Một nguyên tắc hạ tầng đáng nhớ: dùng liên kết tượng trưng (symlink, junction) để tái sử dụng cùng một skill ở nhiều nơi, đừng sao chép — vì “tam sao” là “thất bản”.

Lợi ích thu được

- AI làm việc nhất quán với phong cách đã định ngay cả giữa các phiên khác nhau, mà không cần nhắc lại.
- Context gọn và giàu tín hiệu: mỗi câu ngữ cảnh nạp vào đều có lý do, không còn những đoạn chỉ dẫn treo lơ lửng không dùng đến.
- Tri thức của tổ chức chuyển từ dạng "trí nhớ cá nhân" sang dạng "tài sản có thể tái sử dụng", độc lập với người viết ra nó.

Giới hạn còn lại

Skills tối ưu được context của một LLM đơn lẻ, nhưng chúng sống trong biên giới của chính mô hình đó. Khi tác vụ đòi hỏi thao tác với thế giới bên ngoài — trình duyệt, cơ sở dữ liệu, hạ tầng đám mây — AI vẫn bị giới hạn. Đây chính xác là lúc nguyên lý tỏ-mờ (Fuzzy-Front / Deterministic-Back) đòi ta phải hành động: phân nào cần thi hành chắc chắn, đẩy nó xuống cho code xác định xử lý. Nhưng để AI và code nói chuyện được với nhau một cách hiệu quả, cần một lớp adapter chuẩn hóa. Đó là lý do Bài 3 phải xuất hiện.

Bài 3 — Công cụ ngoài (MCP): cầu nối giữa phần mờ và phần rõ

Bối cảnh & Vấn đề

Không có MCP, AI bị giam trong ngôn ngữ. Nó có thể đọc, suy luận, soạn thảo — nhưng không thể mở trình duyệt để kiểm tra giao diện mình vừa viết, không thể truy vấn cơ sở dữ liệu để hỏi dữ liệu thật, không thể đẩy code lên repository hay tìm kiếm tài liệu ngoài context window. Mọi kiểm chứng đều phải đi vòng qua con người.

Nhưng muốn kết nối AI với bất kỳ công cụ nào, ta phải tự tạo một “phiên dịch viên” hoạt động tại ranh giới tỏ-mờ: ép AI xuất ra output có cấu trúc rồi tự viết code map output đó vào lời gọi hàm của từng hệ thống. Mỗi công cụ một lớp mapping riêng. Thêm công cụ mới là thêm một layer dễ vỡ; đổi mô hình là sửa lại toàn bộ; thêm một team mới là lại có một chuẩn mới không tương thích.

Điều này dẫn tới một vấn đề dài hạn: hệ sinh thái công cụ bị lãng phí, phân mảnh, và không đồng nhất. Cùng một tác vụ có thể có 10 công cụ có thể kết nối với AI, mỗi cái có 1 yêu cầu khác nhau, ưu nhược điểm khác nhau, phù hợp trong những điều kiện khác nhau, nhưng cuối cùng đa phần cái được chọn là cái thứ 11 — tự viết mới.

Tư duy & Nguyên lý

Giải pháp đi theo một tư duy rất cổ điển trong khoa học máy tính: nếu nhiều bên cần nói chuyện với nhau, hãy định nghĩa một giao thức chung thay vì ghép nối đôi một. Giao thức đó đứng ở tầng giữa: phía trên là các AI-agent bất định, phía dưới là các công cụ xác định. Giao thức chính là adapter kinh điển giữa hai miền.

Nhìn qua lăng kính ba nguyên lý nền, đây là hiện thân rõ nhất của nguyên lý tỏ-mờ. AI đứng ở tầng phán đoán: nó quyết định "có nên chụp màn hình bây giờ không, để kiểm cái gì". Công cụ đứng ở tầng thi hành: nhận yêu cầu đã rõ ràng → mở URL → chụp → trả về bitmap. Giao thức chỉ làm một việc: khớp giọng nói của hai bên.

Đây cũng là ví dụ của thin harness: giao thức không tự mọc thêm logic, nó chỉ chuẩn hóa hình dạng của lời yêu cầu và hình dạng của câu trả lời. Mọi tri thức về "khi nào dùng công cụ này" nằm ở skill và ở agent, không nằm ở harness. Đó chính là xuất phát điểm dẫn Anthropic công bố Model Context Protocol (MCP) vào cuối năm 2024 — một chuẩn mở để bất kỳ AI nào cũng có thể đọc, gọi, và nhận kết quả từ bất kỳ công cụ nào — mà không ai phải tự viết lớp mapping riêng nữa.

Cách tiếp cận

Một giao thức công cụ trưởng thành thường có bốn đặc trưng. Một, nó độc lập với nhà cung cấp cụ thể — cùng một giao thức có thể mô tả công cụ từ nhiều bên khác nhau. Hai, nó khai báo rõ "công cụ nào có tham số gì và trả về gì" trước khi AI dùng — để AI biết trước năng lực của mình. Ba, nó cho phép cài đặt/gỡ bỏ công cụ mà không phải sửa bản thân của chính agent. Bốn, nó đi kèm cơ chế xác thực để công cụ nhạy cảm (truy cập dữ liệu, thanh toán) có lớp bảo vệ.

Dưới đây là một vài công cụ thường gặp kèm ví dụ minh họa:

Vai trò	Mô tả	MCP Tool	MCP Server
Điều khiển trình duyệt	AI mở URL, click, nhập liệu, chụp màn hình để kiểm định giao diện.	browser_navigate browser_click browser_screenshot	@playwright/mcp (Microsoft)
Truy vấn cơ sở dữ liệu	AI hỏi dữ liệu thực thay vì bịa	query list_tables describe_table	@modelcontextprotocol/ server-postgres (Anthropic)

	— trả về kết quả xác định từ DB.		
Đọc/ghi file hệ thống	AI đọc, tạo, sửa file trong workspace mà không cần shell.	read_file write_file list_directory	@modelcontextprotocol/ server-filesystem (Anthropic)
Thao tác Git & GitHub	AI đọc lịch sử, tạo commit, mở PR, đọc issue trực tiếp.	git_log git_commit create_pull_request	@modelcontextprotocol/ server-github (Anthropic)
Tìm kiếm web	AI tra cứu thông tin ngoài context window, tránh ảo giác.	web_search local_search	@modelcontextprotocol/ server-brave-search (Anthropic)

Nguyên tắc chọn công cụ là vừa đủ: chỉ bật cái bạn thật sự cần trong giai đoạn hiện tại. Mỗi công cụ đang bật đều tiêu tốn một phần attention và một phần rủi ro bảo mật.

Lợi ích thu được

- AI vượt khỏi biên giới văn bản: nó có thể quan sát, thao tác, kiểm chứng kết quả ở thế giới thật.
- Phần thi hành được giao cho công cụ xác định — giảm đáng kể bất định đầu ra trong những bước mà không được phép sai vì xác suất.
- Hệ công cụ có thể mở rộng/tháo lắp linh hoạt mà không đung vào nội tại của agent.

Giới hạn còn lại

Bây giờ chúng ta có một AI biết code, có bộ não mở rộng qua skills, và có cánh tay nối dài qua các công cụ. Đây là một tác nhân làm việc rất mạnh — nhưng vẫn chỉ là một. Khi nhiệm vụ phức tạp lên, context của một agent sẽ phình to nhanh chóng theo số lượng skill và tool mà nó cần dùng: vừa suy luận kiến trúc, vừa viết logic, vừa làm giao diện, vừa kiểm thử... — tất cả trong cùng một khung cửa sổ. Làm càng nhiều, AI càng “ngu” ra.

Lời giải phải đến từ một tầng khác: thay vì làm một agent “siêu to khổng lồ”, hãy tạo nhiều agent chuyên trách, mỗi agent có context riêng và dày skill ở một

mảng. Đó là chuyển động từ Phần I sang Phần II của chúng ta — từ cá nhân sang đội hình.

Phần II — Hợp tác & Quản trị

Ba bài này của Phần II chuyển từ tầng tác nhân sang hệ thống. Ta chia việc cho nhiều agent chuyên trách, đặt một nhạc trưởng để điều phối, và cuối cùng hội tiếp bằng một trí nhớ sống để hệ thống học được từ chính mình. Đây là lúc “harness engineering” bắt đầu trỗi dậy.

Bài 4 — Sub-agent: chia việc cho đội hình AI chuyên trách

Bối cảnh & Vấn đề

Sau Phần I, ta đã có thể tạo nên một agent "làm tất – ăn cả". Nhưng khi công việc đòi hỏi đồng thời nhiều nghiệp vụ, như phán đoán kiến trúc, viết logic nghiệp vụ, làm giao diện, và kiểm thử trong cùng một phiên, ta bắt đầu thấy các triệu chứng quen thuộc: AI quên chỉ dẫn đã nói lúc đầu, mâu thuẫn với chính mình, hoặc sa đà vào chi tiết này mà bỏ sót chi tiết khác. Nguyên nhân gốc không phải AI kém, mà là context đã quá dày và quá tạp.

Giải pháp đến từ cách mà xã hội loài người đã dùng hàng thiên niên kỷ nay: phân công lao động. Ta không bắt một người vừa làm kiến trúc sư, vừa làm lập trình viên, vừa làm tester, vừa làm thiết kế. Ta lập nhóm. Mỗi người một chuyên môn, một ngăn nhớ, một bộ quy tắc hành nghề riêng. Bạn chỉ được làm 1 việc không phải vì bạn không làm được việc khác, mà bởi vì nếu việc nào bạn cũng làm, thì không việc nào tốt cả.

Tư duy & Nguyên lý

Đưa logic phân công ấy vào thế giới AI cho ta một kiến trúc gọn: một agent = một vai trò = một context riêng = một tập skill dày cho vai trò đó. Đây đồng thời là ba điều cùng lúc: context engineering cấp kiến trúc (mỗi agent một context), thin harness (mỗi agent có khung mỏng), và fat skills (tập skill chuyên biệt cho từng vai trò).

Ba vai trò ví dụ điển hình trong bài thực hành đủ để bao phủ phần lớn các chu trình phát triển hiện nay: một vai trò lo logic lõi (xử lý nghiệp vụ, thuật toán), một vai trò lo trải nghiệm người dùng (giao diện, tương tác, thẩm mỹ), và một vai trò lo kiểm định (kiểm thử chức năng, kiểm chứng hành vi). Tên gọi cụ thể không quan trọng — ta có thể gọi là Core/UX/Test, hoặc Engineer/Designer/QA, hoặc bất cứ gì phù hợp với văn hóa đội nhóm. Cái quan trọng là biên giới chức năng được kẻ rõ.

Cách tiếp cận

Mỗi sub-agent được mô tả bằng một tài liệu ngắn nhưng đủ (theo chuẩn là file SKILL.md, được lưu trong thư mục của từng agent). Trong tài liệu ấy có ít nhất bốn phần: phạm vi công việc (cái gì agent này được làm, cái gì không), bộ kỹ năng và công cụ mà nó được phép gọi, các ràng buộc (khi làm việc gì thì phải theo trình tự nào, lưu ý cái gì, cập nhật/đồng bộ cái nào...), và mẫu đầu ra kỳ vọng (định dạng báo cáo, cách đóng dấu PASS/FAIL).

Bản chất là chúng ta chỉ có 1 agent đang đóng nhiều vai, hay thực sự có nhiều agent? Thật ra không quan trọng. Bạn có thể ban ngày làm kỹ sư, ban đêm làm giảng viên cho 1 trường ĐH nào đó, miễn là vai nào cũng phải diễn cho tròn vẹn. Khi một agent nói, nó nói bằng "giọng" của vai trò — điều này nghe có vẻ hình thức, nhưng có tác dụng cụ thể là với nhiệm vụ nào, AI đều biết cụ thể nó là ai và cần nạp những gì vào đầu. Harness cho đa agent vì thế cũng chỉ cần đủ để định tuyến thông điệp chứ không cần dày.

Lợi ích thu được

- Context của từng agent gọn hẳn so với một "agent làm tất": agent logic không phải chứa chuẩn màu sắc giao diện, agent UX không phải nhớ lịch sử các thuật toán.
- Chất lượng đầu ra tăng lên rõ rệt do từng agent làm việc trong một không gian đã được định hình cho đúng loại suy luận nó cần.
- Mở rộng tùy ý: dự án của bạn có thể phình to thoải mái bằng cách “tuyển thêm” nhân sự, quản trị hợp lý, chứ không phải bắt 1 tác nhân làm thêm 3-4 vai trò.

Giới hạn còn lại

Chúng ta có ba chuyên gia, nhưng chưa chuyên gia của chuyên gia. Bạn phải quyết định "nhiệm vụ này giao cho ai trước, ai sau", nghiệm thu tổng thể, dàn xếp khi hai agent có ý kiến xung đột. Khi tự gọi từng agent một cách thủ công, ta chỉ đang phân mảnh công việc chứ chưa tự động hóa phối hợp. Nút thắt tiếp theo chính là cần một vai trò đứng trên, đóng vai trò nhạc trưởng. Đó là lý do Bài 5 ra đời.

Bài 5 — Agent Manager và khung quản trị

Bối cảnh & Vấn đề

Bạn biết lý do trong 1 dàn nhạc phải có nhạc trưởng, dù mỗi nghệ sĩ có giỏi đến đâu rồi đấy. Ba sub-agent ở Bài 4 cũng vậy. Khi người dùng muốn thêm tính năng X — ai là người đọc yêu cầu, phân rã nó thành các tác vụ cho từng sub-agent, quyết định thứ tự, kiểm tra đầu ra giữa các bước, và đóng dấu hoàn tất ở cuối?

Bạn tự làm? Được thôi, nhưng có chắc cách bạn chọn đã tối ưu chưa? Có theo tiêu chuẩn ngành chưa? Có ai làm vậy không? Có nguy cơ gì sau này muốn phát triển thêm phải đập đi xây lại không? Dự án thay đổi người hoặc bản thân bạn vài tuần sau quay lại làm tiếp có chắc nắm cùng 1 luồng tư duy và cách làm chuẩn, nếu đã từng đạt được, hay không?

Vậy thì giao cho AI làm, nhưng giao cho sub-agent nào cũng có nguy cơ biến nó thành 1 “god-agent” cả. Vì ngay khi một sub-agent ôm thêm việc điều phối, context của nó lại phình ra, và ta lại quay về vấn đề cũ. Rõ ràng, ta cần một vai trò mới tách biệt — chuyên nghề điều phối, không tự mình viết code, không tự mình làm giao diện, không tự test. Việc gì cũng biết ở 1 mức độ nhất định, nhưng không được phép tự tay làm gì cả.

Tư duy & Nguyên lý

Agent Manager là hiện thân kinh điển nhất của thin harness trong kiến trúc của ta. Vai trò này cố ý mỏng: nó không tích lũy logic nghiệp vụ, không giữ bí mật, không tự ra quyết định sáng tạo. Việc của nó là nhận yêu cầu, đọc quy tắc hệ thống, lên kế hoạch thực thi, gọi đúng sub-agent, theo dõi trạng thái, và nghiệm thu cuối cùng.

Manager cũng chính là một resolver cấp cao: nhìn vào yêu cầu, nó quyết định gọi sub-agent nào, nạp những tài liệu nào vào ngữ cảnh của từng sub-agent. Nếu tầng skill ở Bài 2 là resolver trong phạm vi một mô hình, thì Manager là resolver trong phạm vi cả hệ thống.

Cùng với Manager, ta cần hai tài liệu quản trị. Một tài liệu mô tả "ai có mặt trong hệ này, họ làm gì" — đóng vai trò là bản đồ vai trò. Một tài liệu khác mô tả "những nguyên tắc mà cả hệ thống phải tuân theo" — các quy tắc về nghiệm thu, rollback, tài liệu hóa, v.v. Hai tài liệu này là khung quản trị (governance), và đây là nơi harness engineering thực sự hiện hình.

Cách tiếp cận

Một luồng ví dụ điển hình trong bài thực hành mà Manager điều phối có hình dáng sau: nhận yêu cầu nghiệp vụ → phân tích tác động (phần nào của hệ thống sẽ bị chạm tới) → lập kế hoạch thực thi → phân việc cho sub-agent logic → chuyển sang sub-agent giao diện → chuyển sang sub-agent kiểm định → nhận kết quả PASS/FAIL → đóng dấu hoàn tất hoặc yêu cầu rollback. Mỗi bước đều để lại dấu vết trong log, đủ để người đọc về sau tái hiện lại được toàn bộ quá trình.

Trên đây tất nhiên không phải là luồng quy trình cứng — nó chỉ là hình dạng điển hình. Với mỗi loại yêu cầu, Manager có thể tùy chỉnh luồng cho phù hợp. Cái không thay đổi là nguyên tắc: không có sub-agent nào được đứng ra tuyên bố hoàn tất khi chưa qua kiểm định; nếu kiểm định thất bại, phải có cơ chế rollback về trạng thái ổn định trước đó. Đây không phải thủ tục rườm rà — đây là cách ta thu hẹp biên độ bất định ở quy mô hệ thống.

Manager làm ít việc hơn các sub-agent. Nhưng chính vì ít, nó giữ được cái nhìn toàn cục, và biết được khi nào, việc gì cần ai làm. Manager không làm sâu, nhưng nhìn rộng. Nó giữ "thin harness", cho phép các sub-agent được "fat skills".

Lợi ích thu được

- Một yêu cầu nghiệp vụ, dù phức tạp, trở thành một lệnh duy nhất cho người dùng. Mọi phân rã và phối hợp đằng sau đều tự diễn ra.
- Mỗi phiên làm việc để lại một vết tích có cấu trúc — trở thành nguyên liệu cho Bài 6.
- Khung quản trị biến các quyết định thiết kế thành hiện vật, đọc được bởi cả con người lẫn AI, và tái sử dụng được giữa các dự án.

Giới hạn còn lại

Đến cuối Bài 5, hệ thống đã biết phối hợp. Nhưng nó chưa biết học. Mỗi phiên làm việc vẫn là một phiên "khởi tạo lại": những bài học đắt giá từ hôm trước — những lỗi đã gặp, những cạm bẫy đã rút ra — nếu không được ghi lại một cách có cấu trúc, sẽ bay mất khi phiên kết thúc. Vòng lặp tự cải tiến chưa khép kín. Đó là lý do Bài 6 phải xuất hiện.

Bài 6 — Knowledge Items: trí nhớ sống của hệ thống

Bối cảnh & Vấn đề

Lần đầu tiên một bug lạ xuất hiện, cả đội tốn hai giờ để truy nguyên và sửa. Lần thứ hai, một bug giống y chang trong một dự án khác, cả đội (hoặc một phiên AI mới) lại tốn đúng hai giờ. Lần thứ ba, lại hai giờ nữa. Đây là thảm cảnh quen thuộc, và thiệt hại cộng dồn của nó lớn hơn người ta tưởng: nó không chỉ là thời gian mất đi, mà là sự không-trưởng-thành của tổ chức.

Nguyên nhân gốc không phải thiếu trí tuệ mà là thiếu một cơ chế lưu trữ và hồi tiếp. Kinh nghiệm có nhưng nằm rải rác trong trí nhớ cá nhân, trong các đoạn chat đã bị lùi quá xa, trong commit log mà không ai đọc lại. Thông tin là kỷ niệm, không phải tri thức có thể truy hồi.

Tư duy & Nguyên lý

Đây là bài mà một vòng lặp quan trọng hiện nguyên hình — vòng lặp tự cải tiến. Chu trình cơ bản: hệ thống làm một việc → đọc lại chính kết quả mình làm → chưng cất nguyên nhân của những lần làm chưa tốt → viết lại skill (hoặc ghi Knowledge Item) để lần sau làm tốt hơn. Khi khép được vòng này, tri thức không còn là tài liệu tĩnh — nó trở thành nơi tích lũy học hỏi. Output bất định hôm qua trở thành context ổn định hôm nay. Cùng một mô hình, cùng một con người, nhưng kết quả tháng sau cao hơn tháng trước, vì bản thân hệ thống đã trưởng thành.

Sau mỗi nhiệm vụ có dấu vết đáng lưu (một bug lạ, một quyết định kiến trúc có lý do đặc thù, một pattern code tốn nhiều công debug), Manager đọc lại toàn bộ phiên, chưng cất những điểm đáng nhớ thành một Knowledge Item — một đơn vị tri thức có cấu trúc.

Đây chính là distillation (chưng cất tri thức) ở mức vi mô. Distillation là bước mô hình đọc nhiều nguồn về một chủ thể (ở đây: log từ các sub-agent, đoạn hội thoại với người dùng, mã đã sửa), đối chiếu, và viết ra một hồ sơ có cấu trúc — điều cốt lõi, những thay đổi theo thời gian, các mâu thuẫn, khoảng cách giữa lời nói và hành động, các khoảng trống đáng chú ý. Đây là việc không query database nào tự làm được: nó đòi hỏi phán đoán. Khi hệ thống đi đến mức này, AI không còn là công cụ trả lời mà bắt đầu trở thành một phụ tá phân tích. Khác với một note tự do, KI có bốn phần bắt buộc: vấn đề (hiện tượng quan sát được), nguyên nhân gốc (tại sao nó xảy ra), giải pháp (đã làm gì để khắc phục), và phòng ngừa (nguyên tắc để lần sau không vấp lại). Bốn phần này không phải format cho đẹp — chúng là cấu trúc tối thiểu để một bài học có thể được tái sử dụng.

Cơ chế tái sử dụng nằm ở mục lục và hệ thống tag. Mỗi KI được gắn một vài tag. Khi một phiên mới bắt đầu, agent đọc mục lục, đối chiếu tag của task hiện tại với tag của các KI đã có, và chỉ nạp những KI liên quan vào context. Đây một lần nữa là câu chuyện của “ngân sách chú ý” (attention budget) — không cần nạp nhiều, mà phải nạp đúng.

Cách hệ thống tag hoạt động

Tag không phải phụ liệu trang trí — tag là xương sống của cơ chế truy hồi. Một hệ tag tốt có hai đặc tính trái ngược đồng thời: đủ hẹp để hệ thống có thể lọc chính xác và đủ rộng để có thể bao phủ mọi ý tưởng trong KI.

Đó là lý do tag cần được chuẩn hóa. Nếu để tự do, hệ thống sẽ nhanh chóng có các KI tag "bug", "lỗi", và "errors" và không biết với lỗi nào thì cần lùi ngược lại bài học nào ra đọc. 1 vài nhóm tag phổ biến có thể tham khảo như sau: lỗi-và-debug, giao diện, logic, lưu trữ, hạ tầng, tài liệu, bảo mật... Chúng ta cũng có thể thêm vào một quy tắc đơn giản: ưu tiên tối đa dùng tag cũ, nếu thực sự cần tag mới, hãy thêm vào tập chuẩn trước khi dùng. Điều này sẽ giữ cho resolver luôn ở trạng thái tối ưu.

Danh sách tag cụ thể trong tập chuẩn không phải trọng tâm của tài liệu này. Điều đáng nhớ chính là cơ chế: tag ổn định → mục lục ổn định → truy hồi ổn định → “ngân sách chú ý” được bảo vệ → vị vua “context” được an toàn.

Cách tiếp cận

Ba nguyên tắc vận hành KI đáng ghi nhớ.

- **Một** — chỉ Manager được viết KI. Các sub-agent chỉ báo cáo; việc chung cất và quyết định "cái gì đáng nhớ" là đặc quyền của Manager. Điều này tránh hiện tượng KI bị nhân bản và mâu thuẫn giữa các vai trò.
- **Hai** — KI có Session Protocol ngay lối vào. Mỗi phiên làm việc mới, bước đầu tiên là đọc mục lục KI và nêu rõ trong câu trả lời đầu tiên "đã kiểm tra KI — có/không có KI liên quan đến task này". Đây là việc làm rất nhỏ nhưng giúp biến hệ thống từ "có KI" thành "dùng KI".
- **Ba** — chủ động đề xuất ghi KI. Sau một nhiệm vụ đáng nhớ, Manager không chờ người dùng nhắc; nó tự đề xuất. Nếu chờ nhắc mới ghi, hệ thống sẽ chỉ học được những gì người dùng chủ động ép — và con người vốn dĩ lại rất dễ quên.

Lợi ích thu được

- Hệ thống hết quên: một bài học đã được ghi nhận sẽ không mất đi theo phiên làm việc.
- Tốc độ cộng dồn: mỗi dự án hoàn thành đều làm cho dự án kế tiếp nhanh hơn, vì các nền móng tri thức đã được đặt sẵn.
- Đầu ra bất định hôm qua trở thành context ổn định hôm nay — vòng lặp tự cải tiến khép kín.

Giới hạn còn lại

Đến đây chúng ta đã có một hệ thống biết phối hợp và biết học. Nhưng mọi việc vẫn đang diễn ra trong phạm vi máy của người dùng. Mỗi thay đổi code chưa được ghi vết phiên bản một cách có kỷ luật, chưa có cơ chế bắt tay phối hợp giữa các nhân sự trong cùng dự án, chưa cơ chế bảo vệ khỏi code rác, hay 1 cỗ máy thời gian cho chúng ta tua ngược về “1 giây trước thảm họa”. Một trục quan trọng trong vận hành phần mềm vẫn thiếu: dòng chảy phiên bản. Đó là lý do Bài 7 phải xuất hiện.

Phần III — Vận hành toàn trình

Hai bài cuối khép lại mạch tài liệu. Bài 7 đặt một lớp bảo vệ tự động cho phần thi hành ngoài cùng: dòng chảy phiên bản ở phía local. Bài 8 là phép thử tổng hợp xem toàn bộ kiến trúc đã xây có thật sự hoạt động như một thể thống nhất hay chưa.

Bài 7 — Dòng chảy phiên bản rảnh tay với Git

Bối cảnh & Vấn đề

Mỗi thay đổi code, dù nhỏ, đều cần được ghi vết: ai sửa, sửa cái gì, vì lý do gì, vào thời điểm nào. Đây là điều kiện cần để một đội có thể quay ngược (rollback) khi sự cố xảy ra, để hai người cùng sửa không dẫm lên nhau, và để một người mới vào hiểu được lịch sử dự án. Công cụ cổ điển cho việc này là Git, và việc đồng bộ lên một kho chung (remote) là nhiệm vụ phải làm hằng ngày.

Nhưng khi mỗi thao tác Git đòi hỏi vài câu lệnh, khi người dùng phải tự nhớ chuẩn đặt tên nhánh, tự soạn commit message chẵn chu, tự nhớ "chưa qua kiểm thử thì chưa push" — ma sát dần trở nên đủ lớn để hoặc là mọi người bỏ qua, hoặc là thao tác trở nên cầu thả. Cả hai đều dẫn tới cùng một chỗ: lịch sử dự án không đáng tin.

Lưu ý phạm vi của bài này: chúng ta chỉ bàn dòng chảy phiên bản ở phía client (local) — AI tự động commit, tạo nhánh, đẩy lên server, thay con người gõ lệnh Git. Các tầng phía server (pipeline tự động chạy trên hạ tầng sau khi push, triển khai tự động, môi trường staging) là một nội dung khác, không thuộc phạm vi tài liệu này.

Tư duy & Nguyên lý

Đây là ví dụ trong sáng nhất của Fuzzy-Front / Deterministic-Back. Phần mờ — "thay đổi này có đáng là một commit riêng không, message nên viết gì để ba tháng sau đọc lại vẫn hiểu" — là việc của AI. Phần rõ — ghi vết nhị phân, tạo nhánh, đẩy lên remote, tính hash — là việc của Git. Sai lầm là bắt con người làm phần rõ, vì con người làm phần rõ sẽ nhanh chóng phát ngán.

Đây cũng là nơi thin harness thêm một mảnh ghép quan trọng: một lớp bảo vệ có ràng buộc cứng. Nguyên tắc "không đẩy lên remote khi chưa qua kiểm định" không phải lời khuyên lịch sự — nó phải là một cái công mà cả con người lẫn AI

đều không thể mở nếu điều kiện chưa đủ. Harness tuy mỏng, nhưng phải thật chắc chắn.

Cách tiếp cận

Một dòng chảy tự động có hình dáng cơ bản như sau: Sau khi kiểm định đóng dấu PASS, Manager sẽ soạn “commit message” theo chuẩn quy ước (ví dụ, Conventional Commits — “loại-thay-đổi (phạm vi): mô tả ngắn”) để “người đọc” tương lai nhìn tiêu đề là biết bản chất thay đổi. Một nhánh mới được tạo với tiền tố phản ánh loại thay đổi (tính năng mới, sửa lỗi, cập nhật tài liệu, v.v.), thay đổi được đẩy lên nhánh đó, và người quản trị dự án được hỏi một câu duy nhất: có muốn gộp vào nhánh chính không? Nếu có, việc gộp và dọn nhánh cũng được Manager xử lý hết.

Về công cụ, Git là chuẩn thực tế; để giảm ma sát giao tiếp với kho từ xa, một CLI quản trị kho (ví dụ gh cho GitHub, hoặc công cụ tương ứng của các nền tảng khác) thường được dùng kèm. Điều cần nhớ: đây không phải MCP — không phải lớp adapter trừu tượng như ở Bài 3 — mà là lệnh hệ điều hành thực sự. Harness ở đây do người dùng quy định sẵn trong SKILL.md của Manager, chẳng hạn như: quy ước đặt tên commit, quy ước đặt tên nhánh, và những điều kiện kiểm định phải qua trước khi đẩy lên. Lớp harness này vẫn mỏng, nhưng đủ chắc cho cả hệ thống vận hành theo nó.

Lợi ích thu được

- Người dùng không còn phải gõ một lệnh nào. Ma sát về 0, kỷ luật phiên bản trở thành mặc định.
- Lịch sử dự án trở nên có thể đọc được: “commit message” có cấu trúc, nhánh có ý nghĩa, mỗi thay đổi đều đi qua một cổng kiểm định.
- Rollback không còn là ác mộng — nó trở thành một thao tác có thể tin cậy vì mọi thay đổi đều có đủ metadata để quay lại chính xác.

Giới hạn còn lại

Đến đây, tất cả bảy lớp của kiến trúc đã được đặt xuống: IDE, Skills, MCP, Sub-agent, Manager, KI, và Git workflow phía local. Nhưng ta chưa có bằng chứng là chúng hoạt động ăn khớp với nhau như một thể thống nhất. Ta cần 1 cái gì đó để chứng minh. Đó là lý do Bài 8 phải xuất hiện.

Bài 8 — Thử thách tổng hợp: Capstone nghiệm thu toàn hệ thống

Bối cảnh & Vấn đề

Chúng ta đã có một hệ thống nhiều lớp, mỗi lớp đã được kiểm định riêng, nhưng không đồng nghĩa với việc toàn hệ thống hoạt động đúng. Hai lớp có thể tốt trong phòng thí nghiệm nhưng mâu thuẫn khi đứng cạnh nhau. Một khung quản trị có thể đẹp trên giấy nhưng bị bỏ qua khi có áp lực. Một vòng lặp tự cải tiến có thể rất gọn nhưng lại không khép được khi tình huống vượt khỏi trường hợp huấn luyện.

Trong kỹ thuật phần mềm, kiểm nghiệm ở cấp hệ thống (end-to-end) khác hẳn kiểm nghiệm chức năng. Nó không hỏi "chức năng này có chạy đúng không", mà hỏi "khi có một yêu cầu thực tế, toàn bộ guồng máy có vận hành đúng theo thiết kế hay không". Bài 8 là nơi ta hỏi câu hỏi đó với chính hệ thống vừa xây.

Tư duy & Nguyên lý

Một phép nghiệm thu tổng hợp đi đúng tinh thần sẽ không kiểm "tính năng". Nó kiểm nguyên lý. Nó hỏi: Manager có thật sự đọc khung quản trị trước khi phân công không? Việc phân tích tác động có thật sự diễn ra trước khi code được chạm vào không? Luồng sub-agent có đi đúng thứ tự không? Kiểm định có thật sự là một cổng — tức nếu fail thì rollback diễn ra, chứ không phải bị bỏ qua cho nhanh? Sau nhiệm vụ, có tri thức nào được chung cất không, và tài liệu kiến trúc có được cập nhật nếu có thay đổi về thiết kế hay không?

Cách tự nhiên nhất để hỏi những câu đó là một checklist đi kèm yêu cầu nghiệm thu, kèm tiêu chí về bằng chứng. Checklist không phải thủ tục hành chính — nó là một hình thức harness cụ thể: một bộ ràng buộc mỏng nhưng chắc chắn, buộc hệ thống phải đi qua đầy đủ các cổng. Bằng chứng kèm theo (tên file screenshot kiểm thử, tên file KI đã ghi, hash commit) đảm bảo đây là nghiệm thu thật, không phải tick bừa.

Capstone còn là một phép thử khác, tinh tế hơn: thử xem hệ thống có thật sự "thin harness, fat skills" không. Nếu trong quá trình nghiệm thu ta phát hiện harness phình ra quá mức (quá nhiều luật, quá nhiều bước), hoặc skill quá mỏng (agent không biết xử lý tình huống hơi lệch chuẩn), thì bản thân kiến trúc đang mất cân bằng và tất nhiên cần phải điều chỉnh.

Cách tiếp cận

Trong bài thực hành này, chúng ta sẽ không có cùng 1 yêu cầu, mà mỗi học viên sẽ được cho 1 đề Capstone riêng. Một prompt tổng thể được trao cho

Manager, yêu cầu Manager tự phân tích dự án hiện tại và đề xuất một tính năng đủ phức tạp để bắt buộc cả ba vai trò chuyên trách đều phải tham gia. Cùng prompt, một checklist kiểm soát được giao kèm, để Manager đưa nguyên vào artifact kế hoạch và tick dần theo tiến độ.

Để hình dung, có thể nêu vài ví dụ hạng mục thường xuất hiện trong một checklist Capstone điển hình (danh sách đầy đủ phụ thuộc từng dự án và không nên đóng cứng): đã đọc tài liệu quản trị trước khi giao việc; đã phân tích tác động trước khi chạm code; đã đi đúng thứ tự phối hợp giữa các sub-agent; đã có kết quả kiểm định PASS trước khi Manager đóng dấu hoàn tất...

Mỗi hạng mục quan trọng đều phải đi kèm bằng chứng thực tế thay vì chỉ một dấu tick — đây là cơ chế phân biệt "nghiệm thu thật" với "nghiệm thu hình thức". Tên file screenshot, tên file KI, hash commit — những vết tích cụ thể đủ để một người thứ ba đọc lại báo cáo có thể xác minh.

Lợi ích thu được

- Hệ thống được kiểm ở đúng cấp mà nó vận hành — cấp hệ thống, không phải cấp chức năng đơn lẻ.
- Học viên lần đầu đứng ở vai trò kiến trúc sư trưởng: không còn gõ từng câu lệnh, mà quan sát cả nhà máy vận hành.
- Bản thân Capstone để lại một artifact nghiệm thu có giá trị dài hạn: đây là mẫu (template) cho mọi lần triển khai tính năng lớn về sau.

Từ Capstone sang hành trang

Capstone đóng lại một chu trình, nhưng không đóng lại cả hành trình học tập. Từ đây, mỗi tính năng mới mà học viên giao cho cả “biệt đội” đều có thể đi qua cùng một chu trình đã được kiểm chứng: phân tích tác động → phân công → phối hợp → kiểm định → chung cất KI → đồng bộ phiên bản. Hệ thống càng chạy, Knowledge Base càng dày; Knowledge Base càng dày, tốc độ càng tăng. Đây là “lãi suất kép” ở tầng tri thức.

Tổng kết: Từ tám bài thực hành đến một phẩm chất hệ thống

Nhìn lại mạch chuyện

Nếu rút gọn toàn bộ tám bài về một đường thẳng, nó trông như sau. Phần I đặt nền cá nhân: ta có một IDE thể hệ mới, ta trang bị cho AI một bộ não mở rộng bằng skills, ta nối dài cánh tay của nó ra thế giới ngoài qua một tầng giao thức công cụ. Cuối Phần I, ta có “đặc vụ AI” làm việc hiệu quả. Phần II nâng lên cấp hệ thống: ta chia việc cho một đội hình sub-agent chuyên trách, đặt một Manager làm nhạc trưởng, và cuối cùng thêm một trí nhớ sống để hệ thống học được từ chính mình. Cuối Phần II, ta có một hệ biết phối hợp và biết trưởng thành. Phần III khép vòng vận hành: dòng chảy phiên bản tự động cho phần thi hành, và một phép nghiệm thu tổng hợp để xác nhận cả hệ hoạt động như một thể thống nhất.

Mỗi bước đi đều được dẫn dắt bởi cùng một bộ nguyên lý. Biên độ bất định của LLM được thu hẹp dần: từ việc có Implementation Plan để phê duyệt trước (Bài 1), đến việc chuẩn hóa context qua skills (Bài 2), đẩy phần xác định xuống cho công cụ (Bài 3), phân mảnh context theo vai trò (Bài 4), áp khung quản trị (Bài 5), khép vòng học hỏi (Bài 6), đặt cổng cho dòng chảy phiên bản (Bài 7), và nghiệm thu toàn hệ (Bài 8). Tám bài không phải tám thủ thuật — chúng là tám tầng của cùng một chiến lược.

Trục xương sống đã hiện rõ

Tất cả những gì ta làm đều là ứng dụng có hệ thống của ba chân lý nền: LLM là hệ bất định cần được quản lý biên độ hợp lý; prompt-context-harness là ba thể hệ cộng dồn của kỹ thuật điều khiển đầu vào; AI làm phần mờ ở đầu, code làm phần rõ ở cuối. Và tất cả được dẫn dắt bởi một triết lý kiến trúc: thin harness, fat skills. Harness cần mỏng để không chiếm chỗ trong cửa sổ ngữ cảnh; Skill cần dày để mỗi byte trong context đều thật sự có giá trị.

Sau tám bài, điều quan trọng không phải là bạn đã học dùng bao nhiêu công cụ. Mà là bạn đã dựng được một kiến trúc trong đó: trí tuệ được đặt đúng chỗ, thi hành được đặt đúng chỗ, và tăng trưởng đến từ khả năng hệ thống học lại chính mình. Đó mới là "phẩm chất hệ thống" — thứ không phụ thuộc vào phiên bản model bạn đang dùng.

Mở rộng ra cấp tổ chức

Điều đáng suy nghĩ là triết lý thin harness, fat skills không chỉ áp dụng cho agent. Một tổ chức trưởng thành cũng cần một harness mỏng — cấu trúc điều hành không phình đến mức bóp nghẹt sự linh hoạt. Đồng thời, tổ chức ấy cần fat skills — năng lực cốt lõi thật dày, quy trình thật vững, tri thức ngầm được làm rõ, chuẩn thực hành được chưng cất từ kinh nghiệm, và cơ chế cập nhật những chuẩn ấy theo thực tế mới. Khi có được cân bằng này, tổ chức không còn sống bằng sự anh hùng cá nhân hay các cú bùng nổ ngẫu hứng, mà sống bằng năng lực tích lũy.

Nhìn từ góc này, những gì học viên vừa dựng cho một dự án nhỏ là một mô hình thu nhỏ của cách tổ chức AI-first vận hành. Bài học không nằm ở một dự án cụ thể — nó nằm ở kỷ luật thiết kế mà bạn có thể mang sang bất cứ bối cảnh nào khác.

Lời nhắn cuối

Thứ giới hạn bạn, bây giờ hơn bao giờ hết, không phải là công cụ hay kiến thức chuyên ngành. Đó là khả năng hình dung ra điều đáng làm, và cách tổ chức để đưa AI vào làm cùng. Người nào bước ra khỏi khóa học này với một kiến trúc đã sẵn sàng, và một bộ skills đang dày lên mỗi ngày, sẽ thấy năng suất của mình không tăng tuyến tính mà theo hàm mũ. Đường cong đó, theo thời gian, sẽ tách bạn ra khỏi phần còn lại một cách rõ rệt.

Và đó không phải một lời hứa của khóa học. Đó là hệ quả tất yếu của một kiến trúc đúng.

Phụ lục A — Thuật ngữ cốt lõi

Bảng dưới đây giữ những thuật ngữ được dùng xuyên suốt tài liệu. Cố ý rút gọn: những khái niệm ở đây là những mảnh tư duy, học viên sẽ gặp lại chúng trong mọi tình huống AI-agent, không chỉ trong buổi học này.

Thuật ngữ	Ý nghĩa
Agent	Một thực thể AI có vai trò xác định, có context riêng, có bộ kỹ năng riêng, có khả năng tự lập kế hoạch và hành động trong một phạm vi.
Artifact	Một hiện vật có cấu trúc do AI sinh ra trong quá trình làm việc — ví dụ: kế hoạch thực thi, danh sách việc, báo cáo nghiệm thu. Artifact đọc được bởi cả người và AI, tái sử dụng được.
Attention budget	Giả định rằng sự chú ý (attention) của một LLM là tài nguyên hữu hạn. Đổ quá nhiều thông tin vào context làm loãng chú ý; cần cơ chế nạp đúng thứ vào đúng lúc.
Blast radius	Phạm vi tác động của một thay đổi code: những phần nào của hệ thống có thể bị ảnh hưởng. Phân tích blast radius trước khi sửa là một nguyên tắc kỷ luật.
Context engineering	Kỹ thuật thiết kế, kiểm soát và nạp đúng ngữ cảnh vào LLM tại từng thời điểm. Thế hệ thứ hai của kỹ thuật điều khiển đầu vào.
Distillation (chung cất tri thức)	Quá trình mô hình đọc nhiều nguồn về một chủ thể, đối chiếu, và viết ra một hồ sơ có cấu trúc — điều cốt lõi, mâu thuẫn, khoảng trống đáng chú ý. Vượt ra khỏi "tìm và ghép", đi vào "đọc và hiểu".
Fuzzy-Front / Deterministic-Back	Nguyên tắc phân công: phần mờ đẩy lên trước cho AI, phần rõ đẩy ra sau cho code xác định.
Harness	Bộ khung vận hành bao quanh mô hình. Gọi mô hình theo vòng lặp, quản lý context, kết nối công cụ, áp ràng buộc an toàn.
Harness engineering	Kỹ thuật thiết kế bộ khung vận hành cho nhiều agent cùng hoạt động. Thế hệ thứ ba của kỹ thuật điều khiển.

 KI (Knowledge Item)	Một đơn vị tri thức có cấu trúc gồm bốn phần: Vấn đề, Nguyên nhân gốc, Giải pháp, Phòng ngừa. Là viên gạch của trí nhớ sống.
 MCP (Model Context Protocol)	Một tầng giao thức chuẩn giữa AI và các công cụ ngoài. Ví dụ điển hình của Fuzzy-Front / Deterministic-Back ở cấp kiến trúc.
 Orchestrator / Manager	Vai trò đứng trên, điều phối các sub-agent. Đặc trưng: làm ít, điều phối nhiều, giữ cái nhìn toàn cục.
 Prompt engineering	Kỹ thuật soạn câu lệnh để tối ưu đầu ra. Thệ hệ thứ nhất của kỹ thuật điều khiển đầu vào.
 Resolver	Cơ chế định tuyến ngữ cảnh — quyết định nạp tài liệu/skill/KI nào vào lúc nào, thay vì nạp tất cả.
 Skill	Một năng lực đã được đóng gói — phương pháp, quy trình, heuristic. Khác prompt ở chỗ nó mô tả cách làm một loại việc, không phải một việc cụ thể.
 Sub-agent	Một agent chuyên trách một vai trò hẹp, có context và bộ skill riêng. Ngược với "agent làm tất".
 Thin harness, fat skills	Triết lý kiến trúc: khung vận hành mỏng, bộ kỹ năng dày, do Garry Tan đề xuất.

Phụ lục B — Nguồn tham khảo đọc thêm

Danh sách dưới đây chọn lọc các nguồn chính thống hoặc có ảnh hưởng, phục vụ học viên muốn đào sâu. Đây không phải danh sách "phải đọc" — chúng là những cánh cửa, mỗi cánh dẫn sang một hướng suy nghĩ.

- Garry Tan — "Thin harness, fat skills": góc nhìn sáng lập về triết lý kiến trúc agent trong kỷ nguyên AI.
- Anthropic — tài liệu về thiết kế và đánh giá agent: các nguyên tắc thực hành cho agent trong môi trường sản xuất.
- Google — tài liệu kỹ thuật về IDE thể hệ agent-first: kiến trúc, quy trình artifact, các cơ chế phê duyệt.
- Tài liệu Model Context Protocol (MCP): đặc tả chuẩn và các công cụ tham chiếu.
- Conventional Commits — đặc tả quy ước commit message được sử dụng rộng rãi.
- Tài liệu nội bộ dự án: các file SKILL.md, AGENTS.md, RULES.md, và thư mục knowledge của dự án học viên. Đây là nguồn tham khảo quan trọng nhất — vì chúng phản ánh đúng ngữ cảnh mà học viên đang làm việc.